Week 4 Lab Handout

API Design II — Writing the OpenAPI Contract

Time: 110 minutes **Bring: laptop, ENDPOINT_LIST.md, Swagger Editor tab open**

Before you start: today you're making a promise, not writing a doc

Everything you write today becomes a contract the moment you hand it to your ring partner. Before that handoff, you can change anything freely — nobody has built against it yet. After the handoff, every change costs someone else, not just you. That's not a reason to be slow; it's a reason to be precise the first time.

> **Keep this in mind while you work:** if a field's shape could be guessed two different ways, it isn't done yet. A vague field today is a debugging session for your partner in three weeks.

Part A — Quick Setup (10 min)

1. Open editor.swagger.io in a browser tab — this is where you'll validate your spec as you go, not just at the end.
2. Create a new file, `openapi.yaml`, at your repo root.
3. Start with the skeleton below and keep it open as you work.
4. Open your `ENDPOINT_LIST.md` from Week 3 side by side — every row becomes one `path` entry today.

```
openapi: 3.0.0
info:
title: [Your App Name] API
version: 1.0.0
paths:
# every endpoint from ENDPOINT_LIST.md goes here
```

Part B — Write Your Full OpenAPI Spec (45 min)

For **every** endpoint in `ENDPOINT_LIST.md`, add a complete entry: path, method, parameters (if any), request body (if it writes data), and response schema.

The minimum every endpoint needs

- **A** `summary` — one line, what this endpoint does.
- `parameters` (if it takes query or path parameters) — name, location (`query` or `path`), and a `schema` with a real `type`.
- `requestBody` (if it's a write endpoint) — the exact shape of what the caller sends, with `required` fields marked.
- `responses` — at minimum the success case (`'200'` or `'201'`), with a full `schema` describing every field, its type, and a realistic example.

Vague vs. precise — this is what gets graded

Vague (avoid this)	Precise (aim for this)
`dueDate: string`	`dueDate: { type: string, format: date-time, example: “2026-08-13T10:00:00Z” }`

`A response with no schema, just a description`	`A response with every field typed, plus required: [id, title]`

If you can't tell, just from reading your own schema, what a real response would look like — it isn't precise enough yet.

As you write, keep asking

- Would my ring partner know exactly what to expect without asking me a question?
- Does every field have a type? Every object have its required fields listed?
- Did I give at least one realistic example per schema, not just types?

Part C — Validate in Swagger Editor (10 min)

Paste your spec into Swagger Editor as you finish each section — don't wait until the whole file is done to check for the first time.

- **Red markers in the left-hand editor** mean a syntax error — fix these immediately, they compound fast in YAML.
- **The right-hand preview** should render every endpoint as a clean, expandable block. If a section isn't showing up there, something in its structure is broken even if the editor didn't flag it in red.
- Click into a couple of your own endpoints in the preview and read them as if you'd never seen your app before. Do they make sense on their own?

Part D — Contract Handoff (10 min)

1. Send your finished `openapi.yaml` to your **downstream ring partner** — the team that will consume your API.
2. Receive the `openapi.yaml` from your **upstream ring partner** — the API you'll consume.
3. Confirm receipt both ways before moving to Part E — don't start reviewing a file that's still mid-edit on the other end.

Part E — Review What You Received (25 min)

Read through the contract you received from your upstream partner as if you were about to build against it today — because in a few weeks, you will be.

What to look for

- **Ambiguous fields** — anything without a clear type, or a date/status field with no format or enum.
- **Missing cases** — what happens on a 404? An invalid request? If only the success response is documented, that's a real gap.
- **Naming you don't understand** — a field name that doesn't obviously match what it contains.
- **Anything that doesn't match what you expected** from their Week 2 needs conversation or Week 3 endpoint list.

Write down **3 specific questions or ambiguities** — not “looks good,” and not “I don't understand any of it.” Each one should point at a specific field or endpoint.

Student deliverable

Create two files:

1. **openapi.yaml** — your full contract, validated with no errors in Swagger Editor, covering every endpoint from ENDPOINT_LIST.md.
2. **CONTRACT_QUESTIONS.md** — your 3 questions or ambiguities about the contract you received from your upstream partner.

Commit and push both:

```
git add openapi.yaml CONTRACT_QUESTIONS.md
git commit -m "Week 4: OpenAPI contract + partner contract review"
git push
```

(Still no branch-naming convention to follow — that starts once you're working out of the template repository your instructor hands you.)

This week's deliverable checklist

- [ ] Every endpoint from ENDPOINT_LIST.md has a complete entry in openapi.yaml
- [ ] Every schema has real types, required fields marked, and at least one example
- [ ] Spec validated in Swagger Editor with zero errors
- [ ] Contract sent to downstream partner, contract received from upstream partner
- [ ] 3 specific questions written about the received contract
- [ ] openapi.yaml and CONTRACT_QUESTIONS.md committed and pushed

Rubric (30 pts)

Criterion	Points
Spec is valid (passes Swagger Editor validation)	8
All endpoints from Week 3 are represented	6
Request/response schemas are precise (types, required fields, examples)	10
Thoughtful, specific questions on partner's contract	6

Common problems and quick fixes

Problem	Fix
YAML errors that don't make sense	Check indentation first — YAML is whitespace-sensitive, and a single misaligned line can produce a confusing error several lines away
Not sure what “required” fields to list	Ask: if this field were missing, would the response be meaningless? If yes, it's required
Can't think of a realistic example value	Look at your own app's real data — copy an actual value, don't invent a placeholder like “string”
Partner's contract hasn't arrived yet	Start Part E's checklist against your own spec while you wait — the same ambiguity questions apply to your own work
Only found 1–2 real questions, not 3	Look specifically at error responses and edge cases — most first-draft contracts only document the happy path

Next week: Build begins — turning this contract into real, running code for the first time this semester.